Algoritmos e Estruturas de Dados

2ª Série

(Problema)

Número de ocorrências de k-mers

N. 51447 Ricardo Pinto

N. 51445 José Saldanha

N. 51484 Vasco Pereira

Licenciatura em Engenharia Informática e de Computadores

Semestre de Verão 2023/2024

13/05/2023

Índice

Conteúdo

1. INTRODUÇÃO **2**

2. ANÁLISE DE OCORRÊNCIA DE MENÇÕES EM TWEETS **3**

2.1 ANÁLISE DO PROBLEMA 3

2.2 ESTRUTURAS DE DADOS 4

2.3 ALGORITMOS E ANÁLISE DA COMPLEXIDADE 5

3. AVALIAÇÃO EXPERIMENTAL **5**

4. CONCLUSÕES **10**

REFERÊNCIAS **10**

1. Introdução

Neste relatório, vamos abordar o tema do problema da 2º série: análise de ocorrência de k-mers. Para este problema, pretendeu-se desenvolver uma aplicação que permite analisar as ocorrências de determinados k-mers.

Os principais objetivos do problema é vermos uma listagem com o número de ocorrências de cada k-mer distinto.

O relatório está dividido em 6 partes:

- Aprofundação sobre o que é o problema e como foi abordado;
- Avaliação experimental do problema;
- Conclusões tiradas;
- Referências utilizadas para o relatório.

2. Análise de ocorrência de menções em tweets

Resumindo o conteúdo da secção 2, na secção 2.1 é descrito o problema e a abordagem encontrada para o resolver. Na secção 2.2 apresentam-se as estruturas de dados utilizadas. Por fim, a secção 2.3 contém alguns dos principais algoritmos utilizados, como os mesmos consultam/atualizam as estruturas de dados e a análise da sua complexidade.

2.1 Análise do problema

Para este problema, considerámos que cada ficheiro de *input* era constituído por uma única linha com as diferentes bases nucleotídicas (As, Ts, Cs e Gs).

Para retirar as informações deste ficheiro, precisamos que o programa leia a linha que compõem todo o ficheiro.

Para fazermos a contagem dos k-mers, teremos de implementar uma função que conte. Para isto dividimos a linha dada no *input* em *substrings* que vão desde i até k. Depois de feita a divisão, adicionamos esta *substring* a um mapa em que temos a *string* acompanhada de um número inteiro. Ao adicionarmos a *substring*, aumentamos o valor em 1, significando que existe uma *substring* correspondente àquela no ficheiro de *input*.

Finalmente, acabamos por escrever estes resultados num *output*.

Neste problema, foram feitas 2 implementações: uma com recurso ao *kotlin.collections* e outra sem recurso ao *kotlin.collections*. Na implementação com recurso ao *kotlin.collections* utilizamos o mapa que é fornecido pela coleção e na implementação sem recurso ao *kotlin.collections* utilizamos o *HashMap* desenvolvido na parte 3 desta série.

2.2 Estruturas de Dados

As estruturas de dados utilizadas para o problema foi o mapa e o HashMap. O mapa é uma estrutura de dados em que os dados são armazenados segundo o formato de uma chave e um valor correspondente. Na figura 1, podemos ver a sua realização.

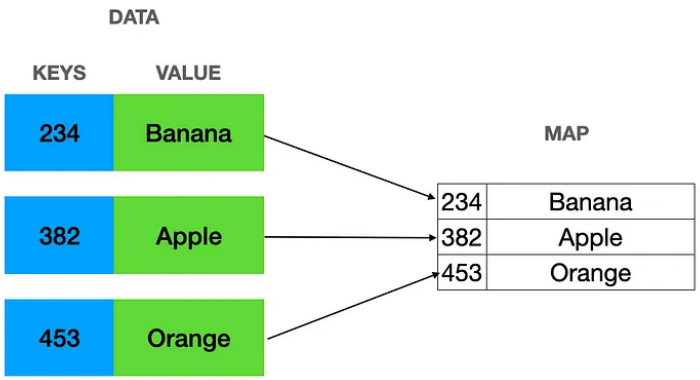


Figura 1: Exemplo da estrutura de dados mapa.

Na figura 2 podemos ver como é realizado um HashMap.

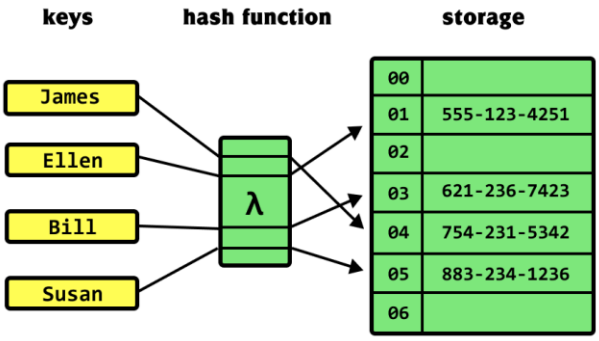


Figura 2: Exemplo da estrutura de dados HashMap.

2.3 Algoritmos e análise da complexidade

O algoritmo é feito logo no início quando temos a função, importada do java, em que vamos ler as linhas do ficheiro. Esta função tem uma complexidade temporal e espacial de *O(n)*, em que n é o número de linhas que o ficheiro de *input* tem, logo para este caso a complexidade temporal e espacial é *O(1)* já que só temos uma linha de texto.

Depois disto feito, entramos no algoritmo da contagem dos k-mers distintos. Neste algoritmo, criamos um mapa de *String* e *Int*, onde vamos armazenar as contagens de cada k-mer que aparecer. De seguida entramos num ciclo *for*, onde dividimos a linha do *input* em *substrings* de i a i+k, e, depois, adicionamos ao mapa, que verifica que se não estiver no mapa mete um valor predefinido, 0, somando 1, no caso da primeira implementação. Para a segunda implementação, fazemos a verificação do valor obtido através da função *get*, aferindo se é nulo ou não. Se for nulo, então adicionamos ao mapa, com a *substring* e o valor 1, se o valor obtido de *get*, é diferente de nulo, adicionamos mais 1 ao valor correspondente da *substring*. Estes dois algoritmos, apesar de serem desenvolvidos de forma diferente, têm a mesma complexidade temporal e espacial. A complexidade temporal é *O(n-k+1)*, em que n é o tamanho da sequência de *input* e k é o tamanho do k-mer, contudo, podemos simplificar esta complexidade para *O(n)*. A complexidade espacial é *O(m)*, uma vez que o espaço ocupado depende do número de k-mers únicos na sequência de *input*.

Depois de acabar a contagem dos k-mers únicos, vamos escrever os resultados num ficheiro. Esta escrita é feita sob um ciclo for, em que a cada elemento do mapa ele escreve no ficheiro. Logo podemos ver que as complexidades temporal e espacial dependem do número de elementos do mapa, ou seja, temos uma complexidade temporal e espacial de *O(m)*, em que m, como dito anteriormente, é o número de k-mers únicos no mapa.

3. Avaliação Experimental

Para a avaliação experimental do problema, usámos o computador do Ricardo que é um portátil HP com um processador Intel i7 da 12ª geração e com 16 GB de RAM. Foram feitos 2 testes, cada um para cada implementação fazendo 5 testes para cada ficheiro, com diferentes quantidades de caracteres e para cada mudança de k a testar e obtivemos os seguintes resultados:

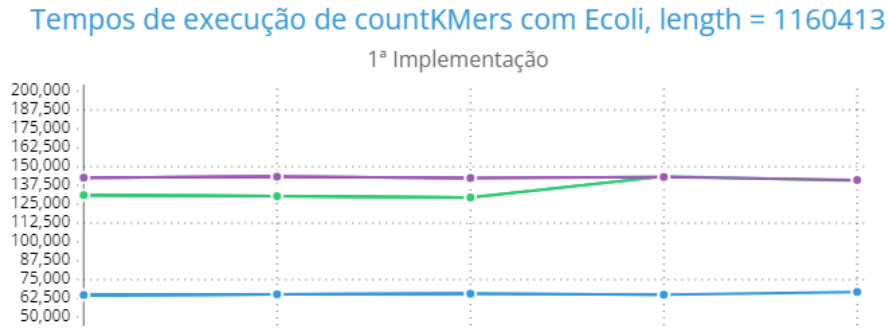
3.1 1ª Implementação

3.1.1 countKMers - ecoli

	Ecoli, length = 1160413					Ecoli, length = 4641652				
K=10	64707	65289	65523	65058	66846	102021	101083	102019	101816	101684
K=100	131116	130337	129457	143420	140734	538070	519829	529382	522211	522380
K=500	142702	143256	142435	143152	140999	535793	539509	542129	535761	536660

Tabela 1: Resultados do tempo de execução da função *countKMers*, para os diferentes tamanhos.

Com 1160413 caracteres temos, em média, para k=10, 65485 ms, para k = 100, 135013 ms e para k = 500, 142509 ms. Com 4641652 caracteres temos que, em média, para k=10, 101725 ms, para k = 100, 526374 ms e para k = 500, 537970 ms.



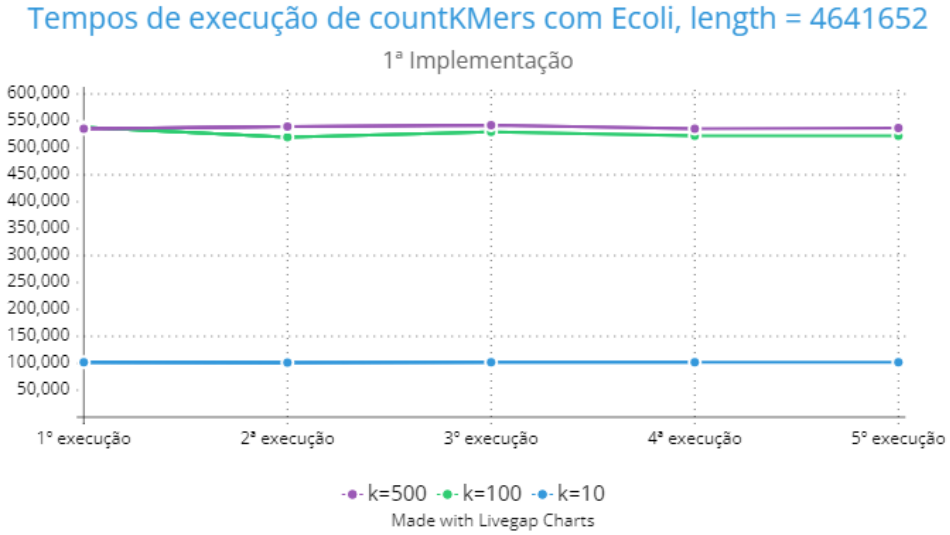
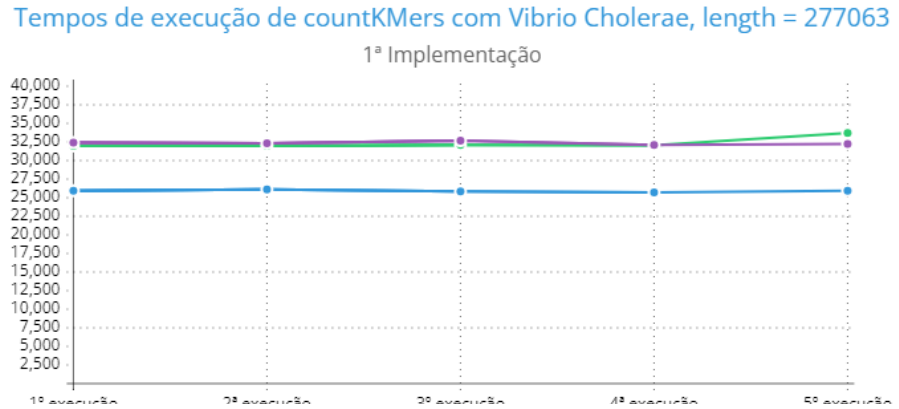


Gráfico 1 e 2: Gráfico 1 correspondente à tabela 1, à parte Ecoli, length = 1160413. Gráfico 1 correspondente à tabela 1, à parte Ecoli, length = 4641652.

3.1.2 *countKMers – Vibrio Cholerae*

	Vibrio Cholerae, length = 277063					Vibrio Cholerae, length = 554126				
K=10	25924	26102	25850	25724	25924	43436	43661	43620	43303	43300
K=100	31962	31983	32086	32071	33687	64021	63813	63225	62715	63276
K=500	32391	32336	32657	32084	32254	64010	63397	64017	64747	64462

Tabela 2: Resultados do tempo de execução da função *countKMers*, para os diferentes tamanhos. Com 277063 caracteres temos, em média, para k=10, 25905 ms, para k = 100, 32358 ms e para k = 500, 32344 ms. Com 554126 caracteres temos que, em média, para k=10, 43464 ms, para k = 100, 63410 ms e para k = 500, 64127 ms.



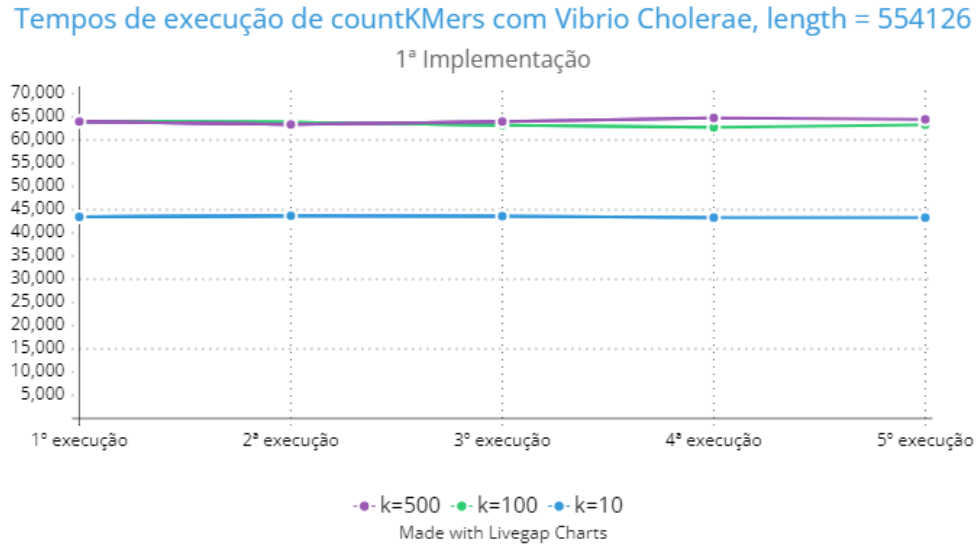


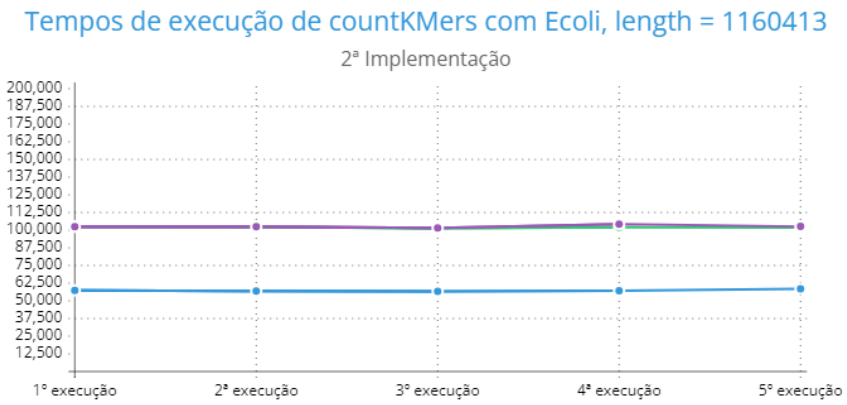
Gráfico 3 e 4: Gráfico 3 correspondente à tabela 2, à parte Vibrio Cholerae, length = 277063. Gráfico 4 correspondente à tabela 2, à parte Vibrio Cholerae, length = 554126.

3.2 2ª Implementação

3.2.1 countKMers – ecoli

	Ecoli, length = 1160413					Ecoli, length = 4641652				
K=10	57335	56968	56671	57226	58404	102128	102989	104101	104239	104419
K=100	102464	102167	101391	102025	101742	404866	431015	345338	423453	394571
K=500	102555	102466	101656	104244	102567	401588	445944	383994	439974	418238

Tabela 3: Resultados do tempo de execução da função *countKMers*, para os diferentes tamanhos. Com 1160413 caracteres temos, em média, para k=10, 57321 ms, para k = 100, 101958 ms e para k = 500, 102698 ms. Com 4641652 caracteres temos que, em média, para k=10, 103575 ms, para k = 100, 399849 ms e para k = 500, 417948 ms.



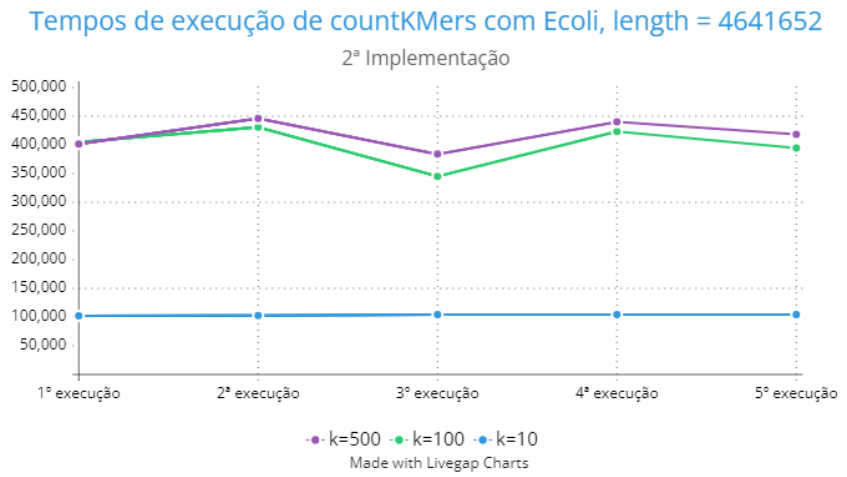


Gráfico 5 e 6: Gráfico 5 correspondente à tabela 3, à parte Ecoli, length = 1160413. Gráfico 6 correspondente à tabela 3, à parte Ecoli, length = 4641652.

3.2.2 *countKMers – Vibrio Cholerae*

	Vibrio Cholerae, length = 277063					Vibrio Cholerae, length = 554126				
K=10	18010	17913	17821	17601	17746	29578	28748	29631	29769	29605
K=100	23058	23893	23341	23332	23656	48301	48436	48641	48300	49059
K=500	23292	23606	23454	24052	24184	48436	48527	49650	50671	48592

Tabela 4: Resultados do tempo de execução da função *countKMers*, para os diferentes tamanhos. Com 277063 caracteres temos, em média, para k=10, 17818 ms, para k = 100, 23456 ms e para k = 500, 23718 ms. Com 554126 caracteres temos que, em média, para k=10, 29466 ms, para k = 100, 48547 ms e para k = 500, 49175 ms.

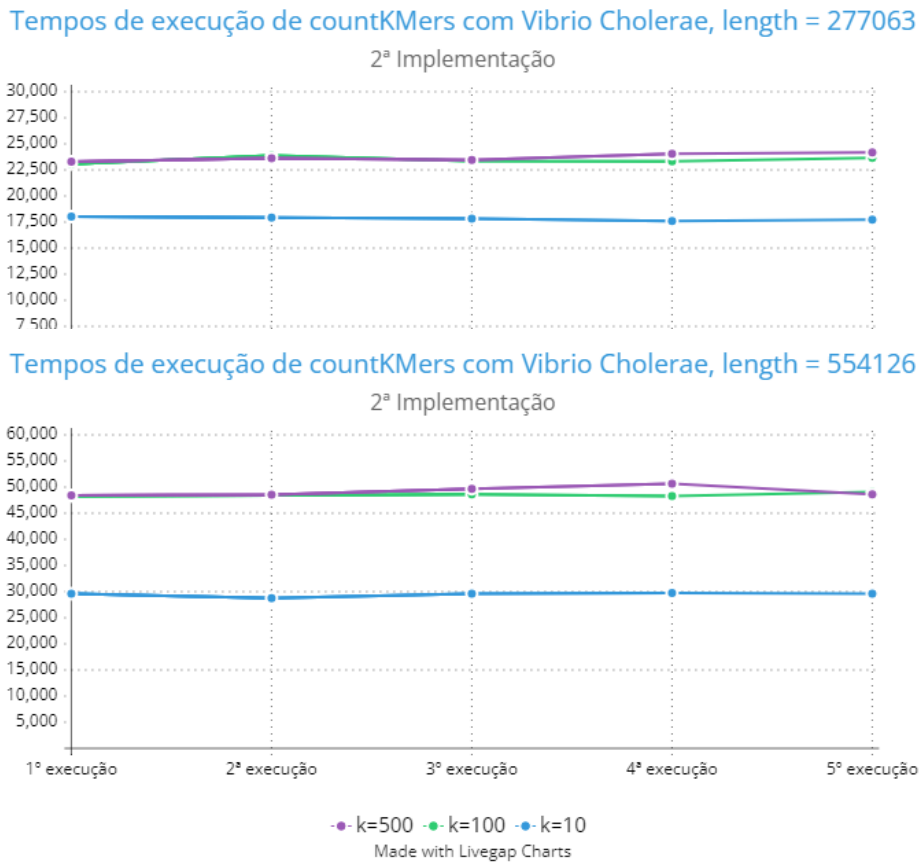


Gráfico 7 e 8: Gráfico 7 correspondente à tabela 4, à parte Vibrio Cholerae, length = 277063. Gráfico 8 correspondente à tabela 4, à parte Vibrio Cholerae, length = 554126.

4. Conclusões

Concluindo, podemos ver através da análise dos tempos de execução que a nossa implementação é uma implementação que parece ser otimizada. Comparando as duas implementações, vimos que a segunda implementação, a que não usamos nada do *kotlin.collections*, foi a implementação que consegui tempos mais rápidos. Isto pode dever-se à maneira de como foi implementado o *HashMap*, que foi reutilizado do exercício 3. Apesar de esta ter sido a maneira mais otimizada que encontramos, tomamos com um grão de sal sobre se esta é a implementação mais otimizada ou não.

Com isto pudemos aprender a como fazer algoritmos utilizando estruturas de dados por nós implementadas, em vez de estruturas de dados dadas pelo *kotlin.collections*.

Referências

[1] "Disciplina: Algoritmos e Estruturas de Dados - 2324SV," Moodle 2023/24. [Online]. Available: https://2324.moodle.isel.pt. [Accessed: 07-04-2024].

[2] Introduction to Algorithms, 3º Edition. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. MIT Press.